

Actividad 1. Introducción a Python.

El siguiente programa consiste en crear un gestor de jugadores de baloncesto. Permite al usuario añadir jugadores, con su nombre, dorsal, canastas de 3, 2 y 1.

Puede ver la cantidad de jugadores que ha creado, sus puntuaciones individuales, cual es el máximo anotador del equipo y el total de anotaciones del equipo por canasta.

Queremos crear un programa que sea bastante escalable, legible y fácil de corregir. Por lo que se han creado 7 funciones.

Funciones:

- Menú:
Esta función enseña al usuario cuales son las acciones que puede realizar. Tenemos una lista de títulos sobre la que iterar, y una variable integer que usaremos para asignar a cada titulo un número. Esto se ha hecho así para que, al añadir cualquier título, este se imprima con su número correspondiente. No puede haber ningún título por encima de “Salir de programa”. Si se pone algo delante de este, dejará de tener el índice 0 para salir del programa.

Actual:

```
=====
Tiene estas opciones para realizar
=====

[1] Introducir un nuevo jugador
[2] Listar jugadores
[3] Máximo anotador
[4] Estadísticas del equipo
[0] Salir del Programa
```

En caso de ampliaciones:

```
=====
Tiene estas opciones para realizar:
=====

[1] Introducir un nuevo jugador
[2] Listar jugadores
[3] Máximo anotador
[4] Estadísticas del equipo
[5] Ampliación
[0] Salir del Programa
```

- Saludo:
Esta función envía un saludo inicial al usuario, donde le solicita el nombre para darle un trato más personalizado. Se pide al usuario si quiere iniciar sesión. Se le añade el método “lower” para que la entrada del usuario siempre sea en minúsculas, aunque escriba todas las mayúsculas. Si el usuario incluye acentos se avisa de que es un valor incorrecto.

```
¡Bienvenido!
Dime tu nombre: CARLOS
¿Desea iniciar sesión, CARLOS? [Si/No]
4
Entrada inválida. Responda 'Si' o 'No'.
¡Bienvenido!
Dime tu nombre: |
```

Una vez el usuario decide que quiere realizar, se entra en un bucle, donde se comprueba que quiere realizar el usuario, rompiendo el bucle True o ejecutando el programa hasta que el usuario quiera salir, con opción “0”. El usuario puede elegir cualquier opción que se le muestre en pantalla por su número. Se han creado 7 funciones en total. En todas las opciones, en caso de no existir jugadores en la lista, se avisa al usuario de que no existen.

- OPCIÓN 1 – Agregar Jugador
El usuario introduce los valores para la creación de un jugador, esta función envía a otra subfunción donde se genera un diccionario para asignar a cada valor. Este diccionario creado se mete en una lista para usos futuros. Se avisa al jugador de que se ha creado el jugador, volviendo al menú.

- OPCIÓN 2 – Listado de jugadores

El usuario con la opción 2 podrá visualizar todos los jugadores que tenga creados. Con sus nombres, dorsales y canastas por tipo. Esto se realiza mediante una iteración sobre cada elemento de la lista creada, puesto que son diccionarios podemos aludir a ellos por la clave para obtener el valor.

- OPCIÓN 3 – Máximo anotador
Con esta función conseguimos decir al usuario cual ha sido el máximo anotador. Primero realizamos por jugador la suma de sus puntuaciones, las guardamos en una lista, y luego extraemos el valor más alto. Posteriormente, comprobamos de nuevo por cada jugador su puntuación total con la puntuación máxima establecida. Si coincide, se imprime el jugador. En caso de que hayan más de 1 jugador con la máxima puntuación, se realiza lo mismo, pero para cada jugador de puntuación máxima.

- OPCIÓN 4 – Estadísticas del Equipo
En esta función creamos una función que nos permite sumar las canastas totales del equipo, al igual que por cada tipo de canasta (de 3, de 2...)
Esto se realiza mediante una iteración por jugador, donde previamente habremos definidos variables globales de esa función donde sumar el valor de cada canasta de cada jugador iterativamente. Con ello, sumando todas obtenemos el total de canastas. Imprimimos estos valores al usuario.

Este bucle de programa permite al usuario salirse en todo momento, además valora ciertas respuestas que el usuario pueda dar sin generar un error y que se le cierre el programa. No he considerado necesario el uso de bloques try-except-else por el hecho de que if-elif-else ya permite realizarlo.